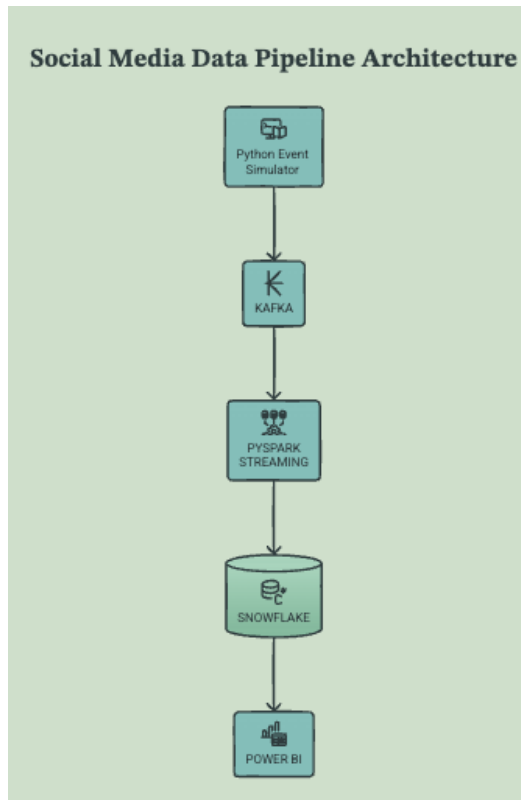


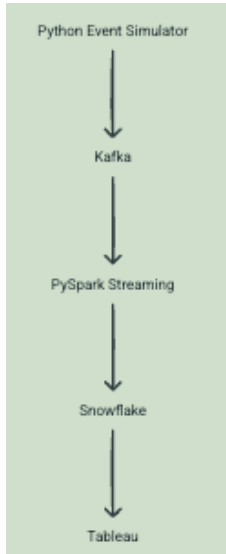
Social_media_analytics_platform

1. Project planning and architecture
 - Architecture diagram



→ Data flow

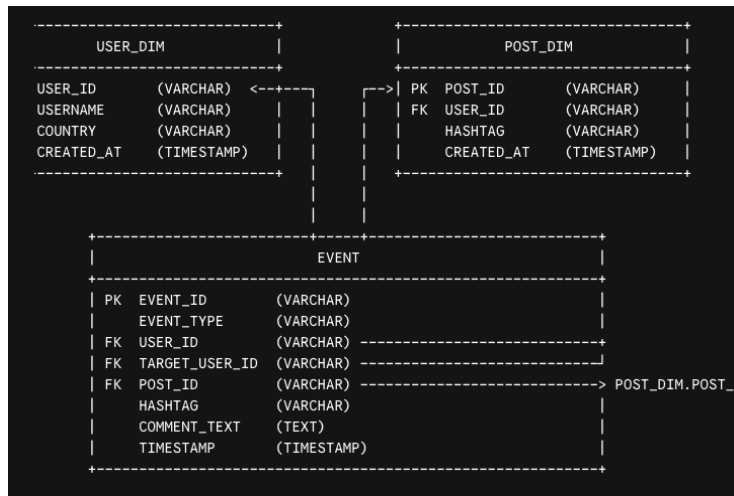
Python creates the events → Kafka streams them → PySpark processes them → Snowflake stores and organizes them → Power BI visualizes the results.



Below table explains what each component does and how the data moves through the architecture.

Component	What it does	Key elements
Python Event Simulator	Generates realistic social media events continuously, simulating activity on a social media platform.	Posts, likes, comments, shares, follows, video views, profile visits
Kafka	Receives and temporarily stores the events as a real-time event stream so they can be processed reliably.	<code>social_media_events</code> topic
PySpark Streaming	Reads the events from Kafka and processes them in real time.	Read from Kafka, parse JSON, validate events, enrich data, aggregate data
Snowflake	Stores the processed data in different layers for storage, transformation, and analytics.	RAW → original incoming data; CURATED → cleaned and transformed data; ANALYTICS → aggregated data ready for reporting
Tableau	Connects to the analytics data in Snowflake and presents the results through dashboards and visualizations.	Engagement, users, posts, trending content, views

- Data model



- Project plan

Step	Project Phase	Main Objective	Success Criteria
1	Planning & Architecture	Define architecture, event types, data model, Snowflake layers, KPIs, and success criteria	Architecture, data model, KPIs, and project plan defined
2	Social Media Event Simulator	Generate users, posts, and realistic events continuously, including viral events	✓ 100+ users ✓ 500+ posts ✓ Realistic continuous events ✓ 1,000+ events/minute
3	Kafka Ingestion	Create Kafka topics and publish and validate simulator events	✓ Kafka successfully receives events
4	PySpark Streaming	Consume Kafka events, parse JSON, validate data, handle errors, and transform events	✓ Real-time stream processing ✓ Malformed events identified/handled
5	Snowflake RAW Layer	Create RAW_EVENTS and load streaming events into Snowflake	✓ Processed events stored in RAW
6	Data Enrichment	Create USER_DIM , POST_DIM , and CURATED_EVENTS	✓ Events enriched and stored in CURATED
7	Trending Hashtag Analytics	Calculate trending hashtags over 1-, 5-, and 15-minute windows	✓ Trending hashtags calculated for all three windows
8	Viral Content Detection	Detect posts with more than 500 likes within 5 minutes	✓ Viral posts detected
9	Influencer Ranking	Rank creators based on likes, comments, and shares	✓ Influencers ranked using engagement

10	Sentiment Analytics	Classify comments as Positive, Neutral, or Negative	✓ Comments classified by sentiment
11	Dashboard Development	Build Tableau dashboards for engagement, viral content, influencers, and other analytics	✓ Tableau displays the resulting analytics

2. Build Social Media Event Simulator

Python simulator

File name: event_simulator.py

```
import json
import random
import time
import uuid
from datetime import datetime, timezone, timedelta

from kafka import KafkaProducer

# =====
# CONFIGURATION
# =====

KAFKA_BOOTSTRAP_SERVERS = "localhost:9092"
KAFKA_TOPIC = "social_media_events"

NUMBER_OF_USERS = 100
NUMBER_OF_POSTS = 500

# 20 events/second = 1,200 events/minute
EVENTS_PER_SECOND = 20

# Post that will be used for viral-content detection
VIRAL_POST_ID = "post_1"

# =====
# USERS / POSTS DATA
# =====

COUNTRIES = [
    "Netherlands",
    "Germany",
    "Spain",
    "France",
    "United Kingdom",
    "Italy",
    "Belgium",
    "Portugal",
    "United States",
    "Canada"
]

HASHTAGS = [
    "#travel",
    "#food",
```

```

    "#fitness",
    "#music",
    "#technology",
    "#fashion",
    "#sports",
    "#gaming",
    "#photography",
    "#lifestyle",
    "#nature",
    "#business",
    "#coding",
    "#movies",
    "#vacation"
]

# Some hashtags are intentionally more popular.
HASHTAG_WEIGHTS = [
    20,
    15,
    14,
    12,
    10,
    8,
    7,
    5,
    4,
    3,
    2,
    2,
    1,
    1,
    1
]

# =====
# EVENT TYPES
# =====

EVENT_TYPES = [
    "post_created",
    "like",
    "comment",
    "share",
    "follow",
    "video_view",
    "profile_visit"
]

EVENT_WEIGHTS = [
    2, # post_created
    35, # like
    10, # comment
    10, # share
    5, # follow
    25, # video_view
    13 # profile_visit
]

# =====
# COMMENT DATA
# Used later for sentiment analysis

```

```

# =====

POSITIVE_COMMENTS = [
    "Amazing!",
    "Love this!",
    "This is fantastic!",
    "Great post!",
    "Absolutely beautiful!",
    "This made my day!",
    "So inspiring!",
    "I really enjoyed this!",
    "Wonderful content!",
    "This is awesome!",
    "Great job!",
    "I love it!",
    "Very helpful!",
    "Fantastic video!",
    "Such a great experience!"
]

NEUTRAL_COMMENTS = [
    "Interesting.",
    "Thanks for sharing.",
    "I see.",
    "Good to know.",
    "Interesting information.",
    "Thanks for posting.",
    "I had not seen this before.",
    "This is useful information.",
    "Nice post.",
    "Good information.",
    "I will keep this in mind.",
    "Interesting perspective.",
    "Thanks for the update.",
    "I understand.",
    "Noted."
]

NEGATIVE_COMMENTS = [
    "I don't like this.",
    "This was disappointing.",
    "Not very useful.",
    "I disagree with this.",
    "This could be much better.",
    "I did not enjoy this.",
    "This is disappointing.",
    "The quality is poor.",
    "I expected more.",
    "Not a good experience.",
    "This did not work for me.",
    "I am not impressed.",
    "Unfortunately this was bad.",
    "I don't recommend this.",
    "This needs improvement."
]

# =====
# KAFKA PRODUCER
# =====

producer = KafkaProducer(
    bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,

```

```

    value_serializer=lambda value: json.dumps(value).encode("utf-8")
)

# =====
# HELPER FUNCTIONS
# =====

def utc_now():
    """Return current UTC timestamp."""
    return datetime.now(timezone.utc).isoformat()

def random_past_timestamp(days=365):
    """Generate a realistic timestamp from the previous year."""

    now = datetime.now(timezone.utc)

    random_seconds = random.randint(
        0,
        days * 24 * 60 * 60
    )

    timestamp = now - timedelta(seconds=random_seconds)

    return timestamp.isoformat()

# =====
# GENERATE USERS
# =====

def generate_users():

    users = []

    for i in range(1, NUMBER_OF_USERS + 1):

        user = {
            "user_id": f"user_{i}",
            "username": f"user_{i:03d}",
            "country": random.choice(COUNTRIES),
            "created_at": random_past_timestamp()
        }

        users.append(user)

    return users

# =====
# GENERATE POSTS
# =====

def generate_posts(users):

    posts = []

    for i in range(1, NUMBER_OF_POSTS + 1):

        post_id = f"post_{i}"

        creator = random.choice(users)

```

```

        # post_1 is always #travel
        if post_id == VIRAL_POST_ID:
            hashtag = "#travel"
        else:
            hashtag = random.choices(
                HASHTAGS,
                weights=HASHTAG_WEIGHTS,
                k=1
            )[0]

        post = {
            "post_id": post_id,
            "user_id": creator["user_id"],
            "hashtag": hashtag,
            "created_at": random_past_timestamp()
        }

        posts.append(post)

    return posts

# =====
# SELECT EVENT TYPE
# =====

def choose_event_type():

    return random.choices(
        EVENT_TYPES,
        weights=EVENT_WEIGHTS,
        k=1
    )[0]

# =====
# GENERATE COMMENT
# =====

def generate_comment():

    sentiment = random.choices(
        ["positive", "neutral", "negative"],
        weights=[45, 35, 20],
        k=1
    )[0]

    if sentiment == "positive":
        return random.choice(POSITIVE_COMMENTS)

    if sentiment == "neutral":
        return random.choice(NEUTRAL_COMMENTS)

    return random.choice(NEGATIVE_COMMENTS)

# =====
# SELECT RANDOM USER DIFFERENT FROM CURRENT USER
# =====

def choose_target_user(users, current_user):

```



```

    target_user = random.choice(users)

    while target_user["user_id"] == current_user["user_id"]:
        target_user = random.choice(users)

    return target_user

# =====
# CREATE EVENT
# =====

def create_event(event_type, users, posts):

    # User performing the event
    user = random.choice(users)

    # Another user, used for follow/profile interactions
    target_user = choose_target_user(users, user)

    event = {
        "event_id": str(uuid.uuid4()),
        "event_type": event_type,

        # -----
        # USER INFORMATION
        # -----
        "user_id": user["user_id"],
        "username": user["username"],
        "country": user["country"],
        "user_created_at": user["created_at"],

        # -----
        # TARGET USER
        # -----
        "target_user_id": None,
        "target_username": None,

        # -----
        # POST INFORMATION
        # -----
        "post_id": None,
        "post_creator_user_id": None,
        "post_created_at": None,
        "hashtag": None,

        # -----
        # COMMENT
        # -----
        "comment_text": None,

        # -----
        # EVENT TIMESTAMP
        # -----
        "timestamp": utc_now()
    }

# =====
# LIKE
# =====

if event_type == "like":

```

```

# 40% of likes go to post_1
if random.random() < 0.40:
    post = posts[0]
else:
    post = random.choice(posts)

creator = next(
    u for u in users
    if u["user_id"] == post["user_id"]
)

event["target_user_id"] = creator["user_id"]
event["target_username"] = creator["username"]

event["post_id"] = post["post_id"]
event["post_creator_user_id"] = post["user_id"]
event["post_created_at"] = post["created_at"]
event["hashtag"] = post["hashtag"]

# =====
# COMMENT
# =====

elif event_type == "comment":

    post = random.choice(posts)

    creator = next(
        u for u in users
        if u["user_id"] == post["user_id"]
    )

    event["target_user_id"] = creator["user_id"]
    event["target_username"] = creator["username"]

    event["post_id"] = post["post_id"]
    event["post_creator_user_id"] = post["user_id"]
    event["post_created_at"] = post["created_at"]
    event["hashtag"] = post["hashtag"]

    event["comment_text"] = generate_comment()

# =====
# SHARE
# =====

elif event_type == "share":

    post = random.choice(posts)

    creator = next(
        u for u in users
        if u["user_id"] == post["user_id"]
    )

    event["target_user_id"] = creator["user_id"]
    event["target_username"] = creator["username"]

    event["post_id"] = post["post_id"]
    event["post_creator_user_id"] = post["user_id"]

```

```

        event["post_created_at"] = post["created_at"]
        event["hashtag"] = post["hashtag"]

# =====
# VIDEO VIEW
# =====

elif event_type == "video_view":

    post = random.choice(posts)

    creator = next(
        u for u in users
        if u["user_id"] == post["user_id"]
    )

    event["target_user_id"] = creator["user_id"]
    event["target_username"] = creator["username"]

    event["post_id"] = post["post_id"]
    event["post_creator_user_id"] = post["user_id"]
    event["post_created_at"] = post["created_at"]
    event["hashtag"] = post["hashtag"]

# =====
# FOLLOW
# =====

elif event_type == "follow":

    event["target_user_id"] = target_user["user_id"]
    event["target_username"] = target_user["username"]

# =====
# PROFILE VISIT
# =====

elif event_type == "profile_visit":

    event["target_user_id"] = target_user["user_id"]
    event["target_username"] = target_user["username"]

# =====
# POST CREATED
# =====

elif event_type == "post_created":

    post = random.choice(posts)

    creator = next(
        u for u in users
        if u["user_id"] == post["user_id"]
    )

    event["target_user_id"] = creator["user_id"]
    event["target_username"] = creator["username"]

    event["post_id"] = post["post_id"]

```

```

        event["post_creator_user_id"] = post["user_id"]
        event["post_created_at"] = post["created_at"]
        event["hashtag"] = post["hashtag"]

    return event

# =====
# MAIN
# =====

print()
print("=====")
print("SOCIAL MEDIA EVENT SIMULATOR")
print("=====")
print()

print("Generating users...")

users = generate_users()

print(f"Generated {len(users)} users.")

print("Generating posts...")

posts = generate_posts(users)

print(f"Generated {len(posts)} posts.")

print()
print("Users and posts are kept in memory for event generation.")
print("No additional JSON files are created.")
print()

print("Sample USER:")
print(json.dumps(users[0], indent=2))

print()
print("Sample POST:")
print(json.dumps(posts[0], indent=2))

print()
print("Starting continuous event generation...")
print()
print(f"Kafka topic: {KAFKA_TOPIC}")
print(f"Events per second: {EVENTS_PER_SECOND}")
print(f"Expected events per minute: {EVENTS_PER_SECOND * 60}")
print(f"Viral post: {VIRAL_POST_ID}")
print()
print("Press Ctrl+C to stop.")
print()

event_count = 0
start_time = time.time()

try:

    while True:

        loop_start = time.time()

```

```

        for _ in range(EVENTS_PER_SECOND):
            event_type = choose_event_type()

            event = create_event(
                event_type,
                users,
                posts
            )

            producer.send(
                KAFKA_TOPIC,
                value=event
            )

            event_count += 1

        producer.flush()

        elapsed = time.time() - start_time

        if elapsed > 0:
            events_per_second = event_count / elapsed
            events_per_minute = events_per_second * 60

        else:
            events_per_second = 0
            events_per_minute = 0

        print(
            f"Events: {event_count:,} | "
            f"Rate: {events_per_second:.2f}/sec | "
            f"~{events_per_minute:.0f}/min"
        )

        loop_elapsed = time.time() - loop_start

        sleep_time = max(
            0,
            1 - loop_elapsed
        )

        time.sleep(sleep_time)

except KeyboardInterrupt:

    print()
    print("Stopping simulator...")

finally:

    producer.flush()
    producer.close()

    print()
    print("Simulator stopped.")
    print(f"Total events generated: {event_count:,}")

```

- Sample event payloads

```
{
  "event_id": "565297ed-82d8-4132-84b4-a1e03e43a89b",
  "event_type": "like",
  "user_id": "user_14",
  "username": "user_014",
  "country": "United States",
  "user_created_at": "2025-11-12T08:28:50.055522+00:00",
  "target_user_id": "user_95",
  "target_username": "user_095",
  "post_id": "post_142",
  "post_creator_user_id": "user_95",
  "post_created_at": "2026-02-13T16:56:22.056312+00:00",
  "hashtag": "#business",
  "comment_text": null,
  "timestamp": "2026-09-03T18:09:07.061426+00:00"
}
```

3. Kafka Ingestion Layer

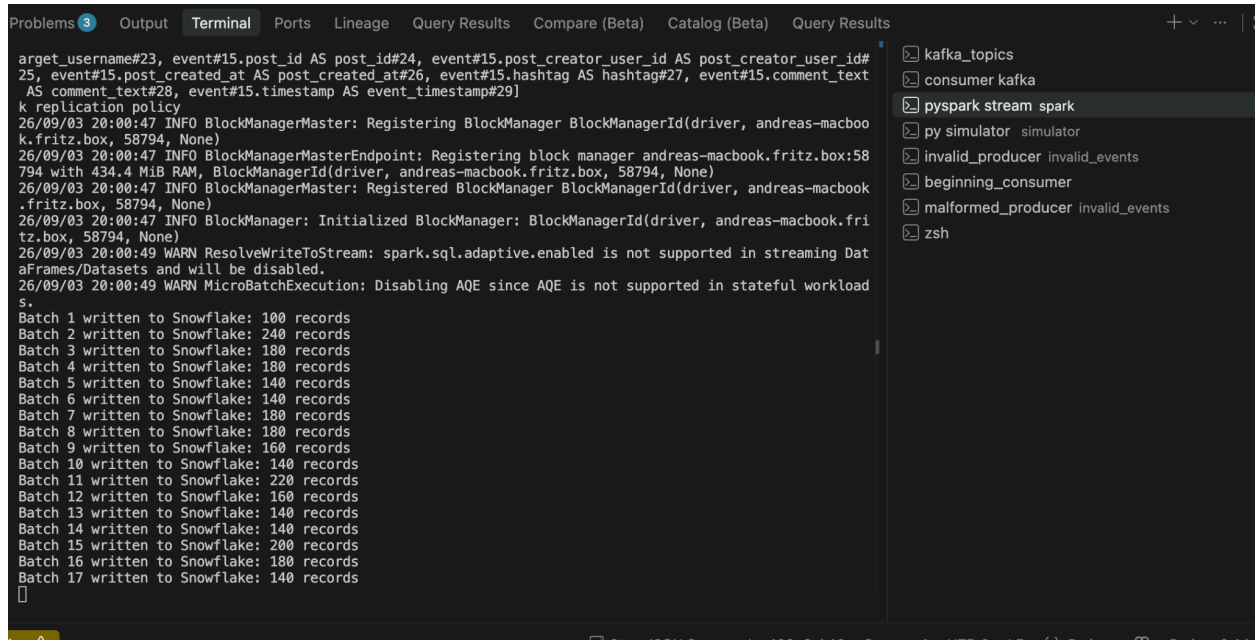
- Working Kafka pipeline



```
{
  "event_id": "e6522b43-4e8a-4acb-b48a-62509da6f4fc", "event_type": "like", "user_id": "user_83", "username": "user_083", "country": "Italy",
  "user_created_at": "2025-10-21T03:52:09.055675+00:00", "target_user_id": "user_26", "target_username": "user_026", "post_id": "post_399", "po
st_creator_user_id": "user_26", "post_created_at": "2026-05-25T23:57:56.057262+00:00", "hashtag": "#music", "comment_text": null, "timestam
p": "2026-09-03T17:55:38.156556+00:00"}
{"event_id": "e89a87f5-441e-4d1e-8dc8-cbd48bdb3703", "event_type": "video_view", "user_id": "user_51", "username": "user_051", "country": "Ge
rmany", "user_created_at": "2025-09-29T01:51:48.055612+00:00", "target_user_id": "user_97", "target_username": "user_097", "post_id": "post_2
70", "post_creator_user_id": "user_97", "post_created_at": "2026-02-14T09:51:51.056803+00:00", "hashtag": "#music", "comment_text": null, "ti
mestamp": "2026-09-03T17:55:38.156670+00:00"}
{"event_id": "51bcc69c-5894-4a83-94d9-bace190c4f79", "event_type": "comment", "user_id": "user_66", "username": "user_066", "country": "Unite
d Kingdom", "user_created_at": "2026-08-14T17:32:12.055633+00:00", "target_user_id": "user_89", "target_username": "user_089", "post_id": "po
st_11", "post_creator_user_id": "user_89", "post_created_at": "2026-04-25T04:19:38.055823+00:00", "hashtag": "#food", "comment_text": "Intere
sting.", "timestamp": "2026-09-03T17:55:38.156789+00:00"}
{"event_id": "5bfb20fb-a7ab-43be-8ff8-97b2a3b2fe13", "event_type": "video_view", "user_id": "user_49", "username": "user_049", "country": "Un
ited Kingdom", "user_created_at": "2026-01-26T00:36:47.055609+00:00", "target_user_id": "user_98", "target_username": "user_098", "post_id":
"post_22", "post_creator_user_id": "user_98", "post_created_at": "2026-01-29T22:41:45.055882+00:00", "hashtag": "#travel", "comment_text": nu
ll, "timestamp": "2026-09-03T17:55:38.156917+00:00"}
{"event_id": "6df1c1de-1494-48d7-81f8-1e4320bd0eff", "event_type": "like", "user_id": "user_60", "username": "user_060", "country": "Canada",
"user_created_at": "2026-06-12T05:32:07.055624+00:00", "target_user_id": "user_45", "target_username": "user_045", "post_id": "post_496", "p
ost_creator_user_id": "user_45", "post_created_at": "2026-06-30T13:46:52.057502+00:00", "hashtag": "#travel", "comment_text": null, "timestam
p": "2026-09-03T17:55:38.157239+00:00"}
{"event_id": "12320f98-ef0f-4baa-bf63-6a1b848cffdb", "event_type": "post_created", "user_id": "user_81", "username": "user_081", "country": "
United Kingdom", "user_created_at": "2025-09-24T20:30:15.055671+00:00", "target_user_id": "user_12", "target_username": "user_012", "post_id":
"post_158", "post_creator_user_id": "user_12", "post_created_at": "2026-05-19T04:08:03.056405+00:00", "hashtag": "#photography", "comment_t
ext": null, "timestamp": "2026-09-03T17:55:38.157506+00:00"}
{"event_id": "0b1fddda-5e2c-4f57-a22c-fd9b22f7eb8c", "event_type": "like", "user_id": "user_23", "username": "user_023", "country": "Germany",
"user_created_at": "2026-04-15T13:20:05.055570+00:00", "target_user_id": "user_11", "target_username": "user_011", "post_id": "post_333", "p
ost_creator_user_id": "user_11", "post_created_at": "2026-03-25T04:11:36.057050+00:00", "hashtag": "#fitness", "comment_text": null, "timest
amp": "2026-09-03T17:55:38.157728+00:00"}
{"event_id": "a640f4c5-cda5-4238-9f6b-4693f3bd61f3", "event_type": "follow", "user_id": "user_76", "username": "user_076", "country": "Unite
d States", "user_created_at": "2025-12-20T00:30:45.055663+00:00", "target_user_id": "user_84", "target_username": "user_084", "post_id": null,
"post_creator_user_id": null, "post_created_at": null, "hashtag": null, "comment_text": null, "timestamp": "2026-09-03T17:55:38.157357+00:00
"}
{"event_id": "f1a92b74-f0b3-4d72-9a27-155b320620c5", "event_type": "post_created", "user_id": "user_69", "username": "user_069", "country": "
Netherlands", "user_created_at": "2026-04-17T18:47:44.055649+00:00", "target_user_id": "user_56", "target_username": "user_056", "post_id": "
post_405", "post_creator_user_id": "user_56", "post_created_at": "2025-12-19T13:48:26.057294+00:00", "hashtag": "#fitness", "comment_text": n
ull, "timestamp": "2026-09-03T17:55:38.157617+00:00"}
{"event_id": "6d77eea5-763f-4055-9bd0-89d2fd9a1ae2", "event_type": "video_view", "user_id": "user_22", "username": "user_022", "country": "It
aly", "user_created_at": "2026-03-06T23:45:34.055569+00:00", "target_user_id": "user_29", "target_username": "user_029", "post_id": "post_285
", "post_creator_user_id": "user_29", "post_created_at": "2025-09-12T12:42:57.056866+00:00", "hashtag": "#fitness", "comment_text": null, "ti
mestamp": "2026-09-03T17:55:38.157827+00:00"}
}
```

4. Real-Time Processing with PySpark

Clean streaming dataset



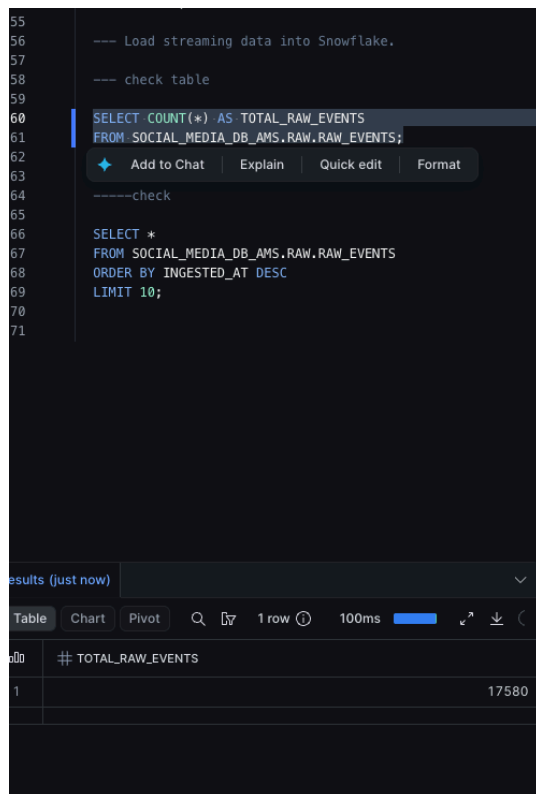
The screenshot shows a Databricks interface with a terminal window displaying logs for a PySpark streaming job. The logs indicate the registration of a BlockManager and the initialization of a streaming query. The right sidebar shows a file explorer with the following items:

- kafka_topics
- consumer kafka
- pyspark stream spark
- py simulator simulator
- invalid_producer invalid_events
- beginning_consumer
- malformed_producer invalid_events
- zsh

```
target_username#23, event#15.post_id AS post_id#24, event#15.post_creator_user_id AS post_creator_user_id#
25, event#15.post_created_at AS post_created_at#26, event#15.hashtag AS hashtag#27, event#15.comment_text
AS comment_text#28, event#15.timestamp AS event_timestamp#29]
k replication policy
26/09/03 20:00:47 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, andreas-macboo
k.fritz.box, 58794, None)
26/09/03 20:00:47 INFO BlockManagerMasterEndpoint: Registering block manager andreas-macbook.fritz.box:58
794 with 434.4 MiB RAM, BlockManagerId(driver, andreas-macbook.fritz.box, 58794, None)
26/09/03 20:00:47 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, andreas-macbook
.fritz.box, 58794, None)
26/09/03 20:00:47 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, andreas-macbook.fri
tz.box, 58794, None)
26/09/03 20:00:49 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming Dat
aFrames/Datasets and will be disabled.
26/09/03 20:00:49 WARN MicroBatchExecution: Disabling AQE since AQE is not supported in stateful workload
s.
Batch 1 written to Snowflake: 100 records
Batch 2 written to Snowflake: 240 records
Batch 3 written to Snowflake: 180 records
Batch 4 written to Snowflake: 180 records
Batch 5 written to Snowflake: 140 records
Batch 6 written to Snowflake: 140 records
Batch 7 written to Snowflake: 180 records
Batch 8 written to Snowflake: 180 records
Batch 9 written to Snowflake: 160 records
Batch 10 written to Snowflake: 140 records
Batch 11 written to Snowflake: 220 records
Batch 12 written to Snowflake: 160 records
Batch 13 written to Snowflake: 140 records
Batch 14 written to Snowflake: 140 records
Batch 15 written to Snowflake: 200 records
Batch 16 written to Snowflake: 180 records
Batch 17 written to Snowflake: 140 records
[]
```

5. Snowflake RAW Layer

RAW layer populated



```
55
56 --- Load streaming data into Snowflake.
57
58 --- check table
59
60 SELECT COUNT(*) AS TOTAL_RAW_EVENTS
61 FROM SOCIAL_MEDIA_DB_AMS.RAW.RAW_EVENTS;
62
63 Add to Chat Explain Quick edit Format
64
65 ----check
66
67 SELECT *
68 FROM SOCIAL_MEDIA_DB_AMS.RAW.RAW_EVENTS
69 ORDER BY INGESTED_AT DESC
70 LIMIT 10;
71
```

The screenshot also shows the results of the query in a table view:

1	TOTAL_RAW_EVENTS
1	17580

6. Data Enrichment

- Enriched curated dataset

SOCIAL_MEDIA_DB_AMS / CURATED / CURATED_EVENTS

Table ACCOUNTADMIN 3 minutes ago 9.9K 590.0KB

Table Details Columns Data Preview Copy History Data Quality Lineage

COMPUTE_WH 100 of 9.9K Rows • Updated just now

	EVENT_ID	EVENT_TYPE	USER_ID	USERNAME	COUNTRY	TARGET_USER_ID	TARGET_USERNAME
1	01bc8ddc-ef86-4038-a27a-ef527f56e7e0	comment	user_10	user_010	Netherlands	user_23	user_023
2	298616f9-d398-4869-a34d-6d8ab56e1cf1	profile_visit	user_60	user_060	Canada	user_72	user_072
3	ab55c091-f840-4634-9674-05da5dec7ad6	like	user_39	user_039	Belgium	user_73	user_073
4	17fb448c-3132-4982-8195-aef12d5a64d7	video_view	user_9	user_009	Spain	user_35	user_035
5	2383f162-59b6-45a8-a07b-3c83c35bb527	like	user_67	user_067	Italy	user_33	user_033
6	2b7efb80-8be6-434e-a4e6-d7adba344be1	like	user_11	user_011	Germany	user_28	user_028
7	ba73b6c6-ad06-4acf-a172-27229e5329b8	profile_visit	user_27	user_027	Belgium	user_21	user_021
8	fa53b4f8-7603-4e43-bdde-ba669c0cdbc8	like	user_16	user_016	Italy	user_42	user_042
9	5248d3c7-2edf-49cd-ba7a-47785a48f467	comment	user_71	user_071	Netherlands	user_38	user_038
10	c6648f2b-014b-4cf1-a851-b14f7d2bdb2d	comment	user_4	user_004	Netherlands	user_5	user_005
11	db896663-34cc-45a8-ac2a-becb1a487f8b	comment	user_8	user_008	Portugal	user_16	user_016
12	0ca25e17-44b5-429a-b6ad-f8c24083a04c	video_view	user_22	user_022	Italy	user_26	user_026
13	fb93374a-0992-4bea-b229-28873407bf19	like	user_16	user_016	Italy	user_39	user_039
14	489f251f-9991-4daa-8946-9770cacabb9c	comment	user_41	user_041	Germany	user_3	user_003
15	52e156d7-1d66-421a-8c91-18ed58ea3218	like	user_84	user_084	Canada	user_74	user_074
16	2b8795d3-bcc5-4fe7-8b3f-301132e4e60b	like	user_82	user_082	France	user_36	user_036
17	e45af1c4-2d97-42c8-b298-6406039ce05e	like	user_67	user_067	Italy	user_14	user_014
18	5d106ca8-bd30-466f-b159-29f844314dce	share	user_46	user_046	France	user_28	user_028
19	8e107479-fef1-4693-a664-d20da28ee405	like	user_44	user_044	France	user_81	user_081
20	247325c4-416b-4dab-a9ee-f09f2bfa8b20	like	user_48	user_048	United Kingdom	user_81	user_081
21	6a5ef98f-b7ba-4404-b41f-17881bcb289d	like	user_87	user_087	Netherlands	user_81	user_081
22	246c6569-2f80-4a19-9c58-d04c8363fd98	like	user_83	user_083	Italy	user_35	user_035
23	78d15a1e-15f3-4f20-ac23-b170ac661f26	share	user_64	user_064	Canada	user_2	user_002
24	1c062033-0d5e-49b4-b3d5-816488e605da	like	user_89	user_089	Germany	user_36	user_036
25	4ef3d54f-8b8d-4257-9fb0-6bb5ad38dc48	video_view	user_4	user_004	Netherlands	user_33	user_033
26	657c39c7-c778-47b2-afb9-c8f73f97b51f	share	user_9	user_009	Spain	user_38	user_038
27	244fa98b-e33d-4993-be30-6bae9767ad10	video_view	user_85	user_085	United States	user_84	user_084
28	d8dfcc08-4a89-40a3-9b0d-baabb4801676	profile_visit	user_17	user_017	Portugal	user_48	user_048
29	6a7d1139-f58d-4730-8f7d-cd258db78f62	like	user_3	user_003	Netherlands	user_73	user_073
30	e5ddf8e-5b78-4a73-97ed-fd06bfb1baac	video_view	user_50	user_050	Germany	user_77	user_077
31	b924a8d0-970a-4856-85d2-493240e8a656	like	user_38	user_038	Belgium	user_9	user_009
32	905466ce-a82f-400c-9577-73464a9a6810	share	user_14	user_014	United States	user_69	user_069

7. Trending Hashtag Analytics

Results (31 minutes ago)Results (just now)

TableChartPivot

Quick insights

28 rows167ms

#	# WINDOW_MINUTES	# RANK	^ HASHTAG	# EVENT_COUNT	🕒 WINDOW_START	🕒 WINDOW_END
1	1	1	#travel	9	2026-09-03 12:38:20.730 -0700	2026-09-03 12:39:20.730 -0700
2	1	2	#food	7	2026-09-03 12:38:20.730 -0700	2026-09-03 12:39:20.730 -0700
3	1	3	#fitness	4	2026-09-03 12:38:20.730 -0700	2026-09-03 12:39:20.730 -0700
4	1	4	#music	2	2026-09-03 12:38:20.730 -0700	2026-09-03 12:39:20.730 -0700
5	1	5	#technology	2	2026-09-03 12:38:20.730 -0700	2026-09-03 12:39:20.730 -0700
6	1	6	#sports	1	2026-09-03 12:38:20.730 -0700	2026-09-03 12:39:20.730 -0700
7	1	7	#lifestyle	1	2026-09-03 12:38:20.730 -0700	2026-09-03 12:39:20.730 -0700
8	1	8	#fashion	1	2026-09-03 12:38:20.730 -0700	2026-09-03 12:39:20.730 -0700
9	5	1	#travel	772	2026-09-03 12:05:24.620 -0700	2026-09-03 12:10:24.620 -0700
10	5	2	#fitness	300	2026-09-03 12:05:24.620 -0700	2026-09-03 12:10:24.620 -0700
11	5	3	#music	243	2026-09-03 12:05:24.620 -0700	2026-09-03 12:10:24.620 -0700
12	5	4	#food	241	2026-09-03 12:05:24.620 -0700	2026-09-03 12:10:24.620 -0700
13	5	5	#technology	193	2026-09-03 12:05:24.620 -0700	2026-09-03 12:10:24.620 -0700
14	5	6	#sports	134	2026-09-03 12:05:24.620 -0700	2026-09-03 12:10:24.620 -0700
15	5	7	#fashion	126	2026-09-03 12:05:24.620 -0700	2026-09-03 12:10:24.620 -0700
16	5	8	#gaming	99	2026-09-03 12:05:24.620 -0700	2026-09-03 12:10:24.620 -0700
17	5	9	#business	75	2026-09-03 12:05:24.620 -0700	2026-09-03 12:10:24.620 -0700
18	5	10	#lifestyle	61	2026-09-03 12:05:24.620 -0700	2026-09-03 12:10:24.620 -0700
19	15	1	#travel	959	2026-09-03 11:55:24.620 -0700	2026-09-03 12:10:24.620 -0700
20	15	2	#fitness	378	2026-09-03 11:55:24.620 -0700	2026-09-03 12:10:24.620 -0700
21	15	3	#food	315	2026-09-03 11:55:24.620 -0700	2026-09-03 12:10:24.620 -0700
22	15	4	#music	302	2026-09-03 11:55:24.620 -0700	2026-09-03 12:10:24.620 -0700
23	15	5	#technology	238	2026-09-03 11:55:24.620 -0700	2026-09-03 12:10:24.620 -0700
24	15	6	#sports	174	2026-09-03 11:55:24.620 -0700	2026-09-03 12:10:24.620 -0700
25	15	7	#fashion	162	2026-09-03 11:55:24.620 -0700	2026-09-03 12:10:24.620 -0700
26	15	8	#gaming	126	2026-09-03 11:55:24.620 -0700	2026-09-03 12:10:24.620 -0700
27	15	9	#business	91	2026-09-03 11:55:24.620 -0700	2026-09-03 12:10:24.620 -0700
28	15	10	#photography	77	2026-09-03 11:55:24.620 -0700	2026-09-03 12:10:24.620 -0700

8. Viral Content Detection

VIRAL_POSTS table

Results (42 minutes ago)Results (just now)

TableChartPivot

1 row152ms

#	^ POST_ID	# LIKE_COUNT	# WINDOW_MINUTES	🕒 WINDOW_START	🕒 WINDOW_END	🕒 VIRAL_DETECTED_AT
1	post_1	747	5	2026-09-03 12:45:16.383 -0700	2026-09-03 12:50:16.383 -0700	2026-09-03 12:50:16.383 -0700

9. Influencer Ranking table

Results (48 minutes ago)Results (just now)

TableChartPivot

Quick insights

10 rows127ms

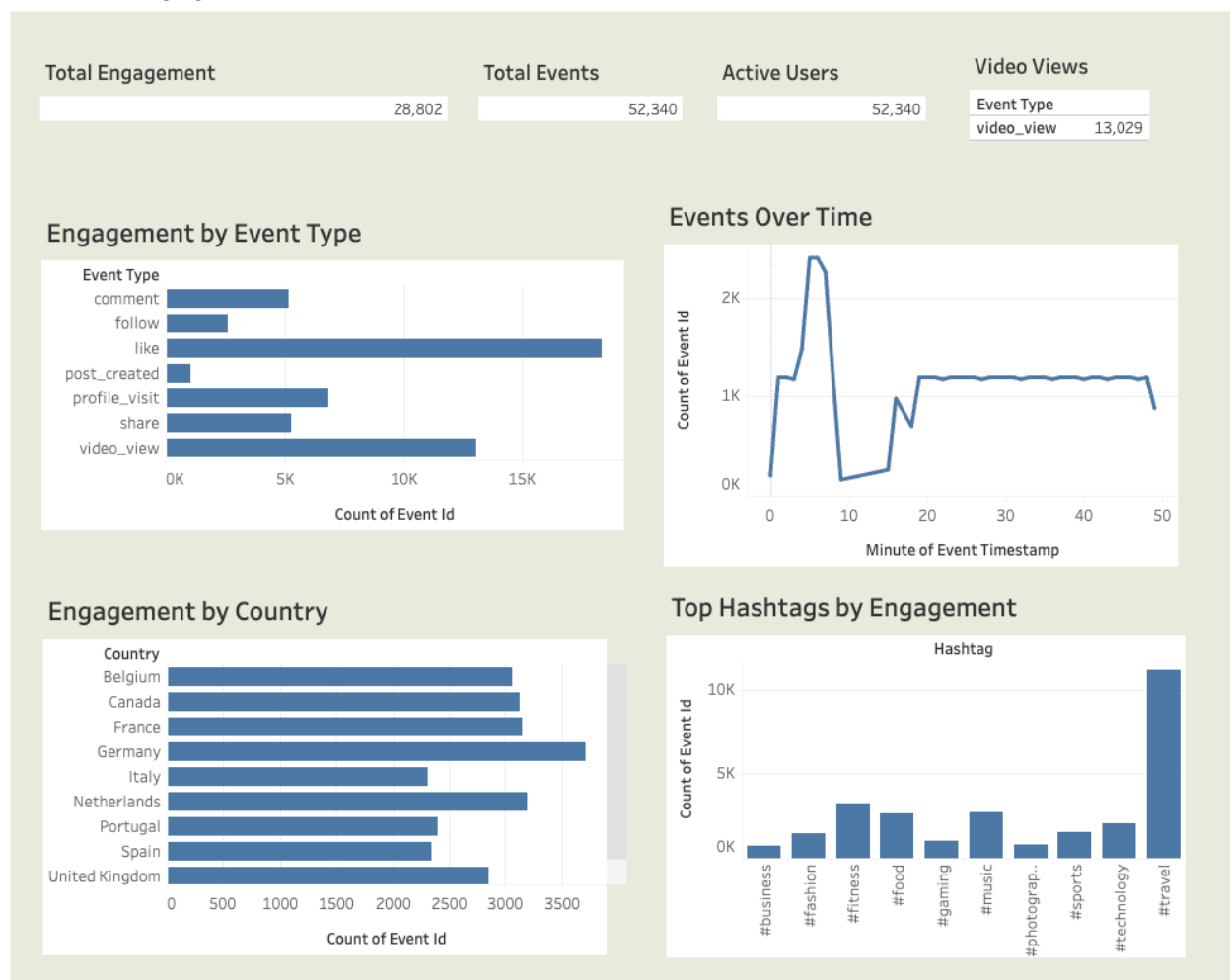
#	^ USER_ID	^ USERNAME	# LIKE_COUNT	# COMMENT_COUNT	# SHARE_COUNT	# TOTAL_ENGAGEMENT	# ENGAGEMENT_SCORE	# RANK	🕒 CALCULATED_AT
1	user_81	user_081	7461	62	51	7574	7738.00	1	2026-09-03 12:56:43.036 -0700
2	user_2	user_002	248	123	122	493	860.00	2	2026-09-03 12:56:43.036 -0700
3	user_46	user_046	230	106	116	452	790.00	3	2026-09-03 12:56:43.036 -0700
4	user_72	user_072	235	93	108	436	745.00	4	2026-09-03 12:56:43.036 -0700
5	user_60	user_060	178	96	104	378	682.00	5	2026-09-03 12:56:43.036 -0700
6	user_96	user_096	197	103	90	390	673.00	6	2026-09-03 12:56:43.036 -0700
7	user_80	user_080	187	98	93	378	662.00	7	2026-09-03 12:56:43.036 -0700
8	user_87	user_087	206	85	84	375	628.00	8	2026-09-03 12:56:43.036 -0700
9	user_9	user_009	197	88	82	367	619.00	9	2026-09-03 12:56:43.036 -0700
10	user_74	user_074	192	79	86	357	608.00	10	2026-09-03 12:56:43.036 -0700

10. Sentiment Analytics

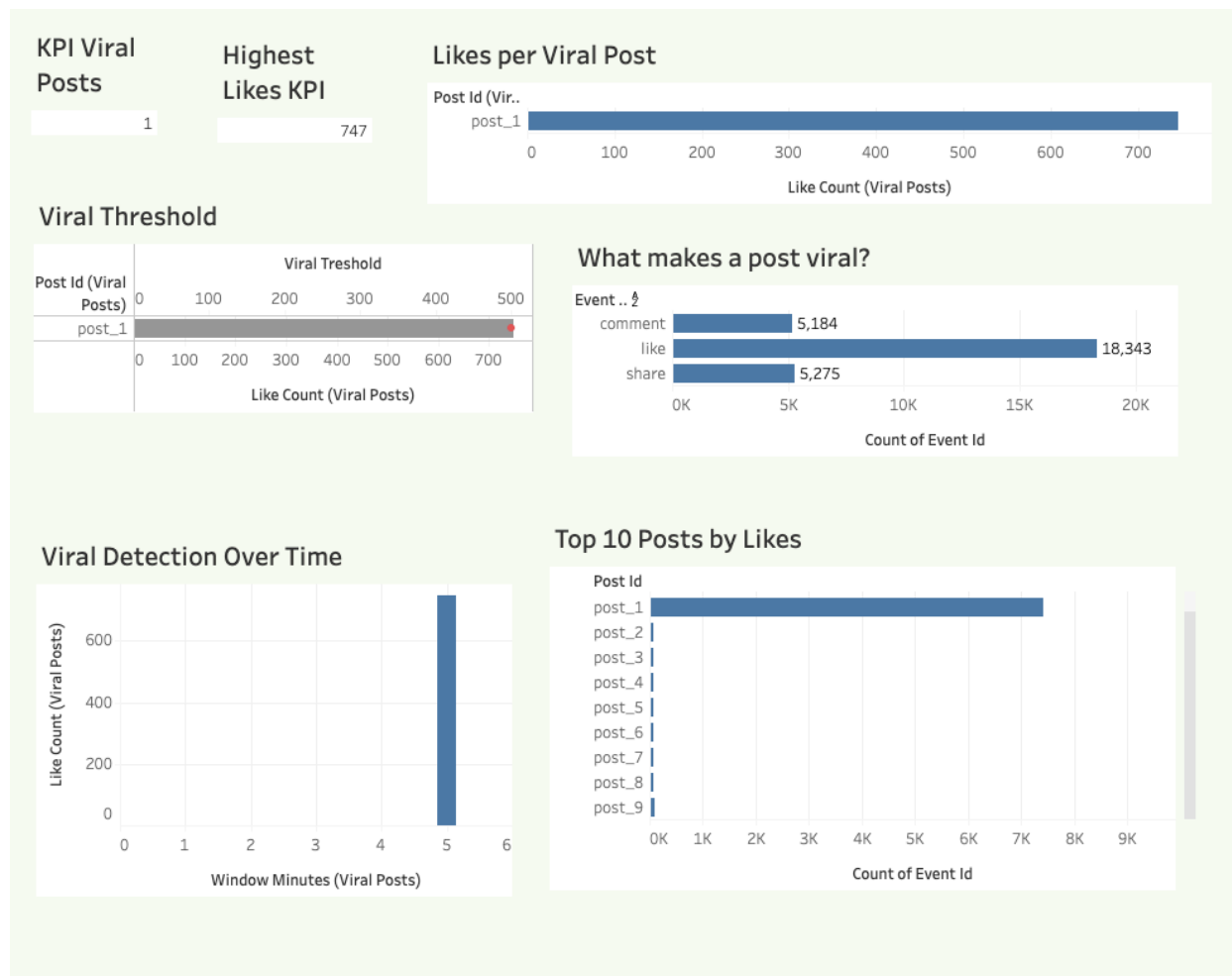
COMMENT_SENTIMENT analytics

Results (54 minutes ago) Results (1 minute ago)			
Table	Chart	Pivot	Quick insights
id	HOURL	SENTIMENT	COMMENT_COUNT
1	2026-09-03 18:00:00.000 +0000	Negative	198
2	2026-09-03 18:00:00.000 +0000	Neutral	364
3	2026-09-03 18:00:00.000 +0000	Positive	454
4	2026-09-03 19:00:00.000 +0000	Negative	864
5	2026-09-03 19:00:00.000 +0000	Neutral	1464
6	2026-09-03 19:00:00.000 +0000	Positive	1840

- Build Engagement Dashboard.



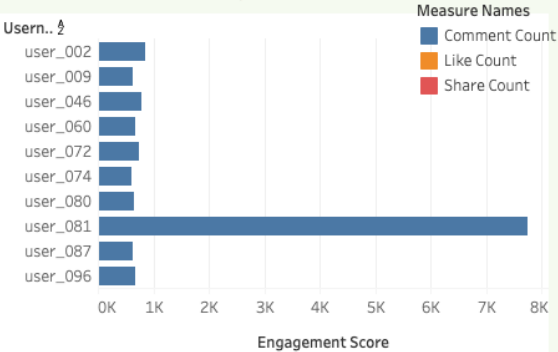
- Build Viral Content Dashboard.



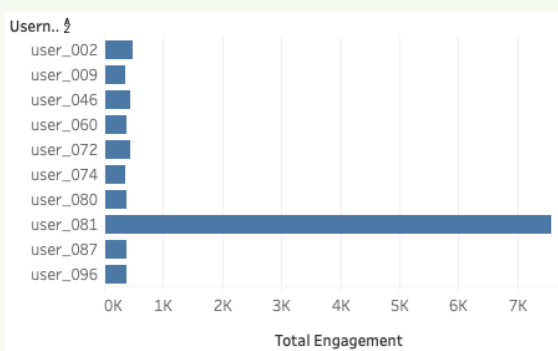
- Build Influencer Dashboard.

Influencer Dashboard

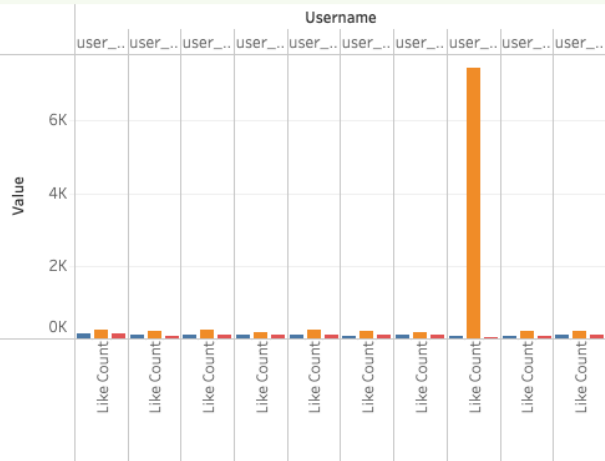
Top 10 Influencers by Engagement Score



Total Engagement by Influencer



Likes, Comments & Shares by Influencer



Influencer Ranking Table

Users	Engage..	Like Co..	Rank	Share C..	Total E..
user_081	7,738	7,461	1	51	7,574
user_002	860	248	2	122	493
user_046	790	230	3	116	452
user_072	745	235	4	108	436
user_060	682	178	5	104	378
user_096	673	197	6	90	390
user_080	662	187	7	93	378
user_087	628	206	8	84	375
user_009	619	197	9	82	367
user_074	608	192	10	86	357
user_005	599	200	11	85	357
user_032	597	171	12	80	344
user_016	583	164	13	81	333